

RAG From Scratch

Wednesday, July 29, 2026 7:55 PM

Agenda:

1. Introduction of RAG
2. RAG Architecture
3. End to End RAG Pipeline
4. Advantage Disadvantage of RAG
5. Build RAG from Scratch
6. Multimodal RAG
7. RAG vs Finetuning
8. Evaluation Of Matrices of RAG
9. How to improve RAG system and important research paper.

Introduction Of RAG: (Retrieval-Augmented Generation)

Retrieval-Augmented Generation (RAG) is an AI architecture that combines the power of:

- **Retrieval** → Finding relevant information from external knowledge sources.
- **Generation** → Using a Large Language Model (LLM) to generate accurate, natural-language answers based on the retrieved information.

In simple terms:

RAG allows an LLM to "look up" information before answering a question.

Instead of relying only on what the model learned during training, RAG retrieves the latest or domain-specific information and uses it to generate a response.

Layman's Example:

Imagine you ask your friend: "**What is the latest policy of my company?**"

Your friend has two choices:

Without RAG

Your friend answers only from memory.

- May remember correctly.
- May forget.
- May give outdated information.

Ex: "I think your company allows only two work-from-home days."

But the policy changed yesterday.

With RAG

Your friend first

- Opens the company handbook.
- Searches for the latest policy.
- Reads the documents
- Gives the answer.

Ex: "According to the HR handbook updated yesterday, employees can work remotely three days a week."

This is exactly how RAG works.

Why do we need RAG?

Large Language Models have several limitations:

1. **Knowledge Cutoff** (Model only knows information available up to its training date.
2. **Hallucination** - (LLMs sometimes confidently generate incorrect information) Ex: "**What is the phone number of XYZ company?**"
The model might invent one.
3. **Private Data** - Companies have (Pdf, Excel, Emails, Reports, Database). The LLM has never seen these during training.
4. **Expensive Retraining** - Updating an LLM every time your docs change is impractical.

Advantages of RAG

- Up-to-date information
- Uses private documents securely
- Reduces hallucinations
- No need to retrain the LLM for every document update
- Scales to millions of docs using vector search
- Supports citations and source references
- Cost-effective compared to fine-tuning for knowledge updates

Real-World Applications

- **Customer Support:** Answer questions from product manuals and FAQs.
- **Enterprise Search:** Search company documents, policies, and reports.
- **Healthcare:** Retrieve medical guidelines before generating responses.
- **Legal:** Search contracts and legal documents.
- **Education:** Build question-answering systems over textbooks and lecture notes.
- **Finance:** Query annual reports, regulations, and compliance documents.

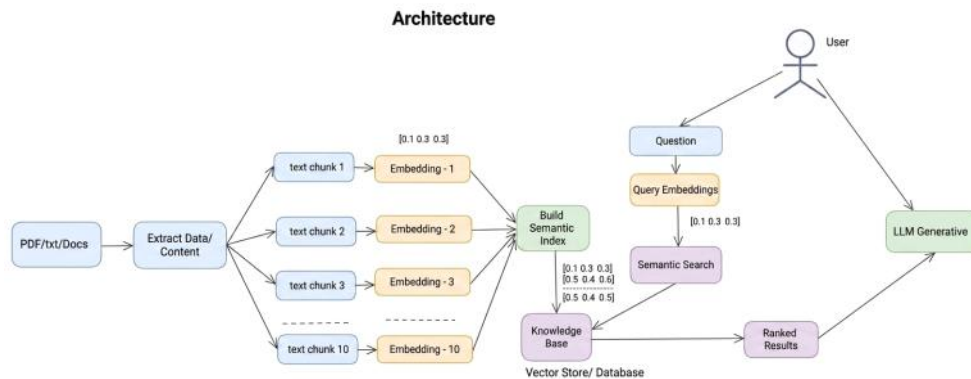
Why do we need RAG?

Large Language Models have several limitations:

1. **Knowledge Cutoff** - The model only knows information available up to its training date.
2. **Hallucination** - LLMs sometimes confidently generate incorrect information.
3. **Private Data** - Companies have (PDFs, Excel files, Emails, Reports, Databases)
4. **Expensive Retraining** - Updating an LLM every time your documents change is impractical.

2. RAG Architecture:

1. Ingestion
2. Retrieval
3. Generation

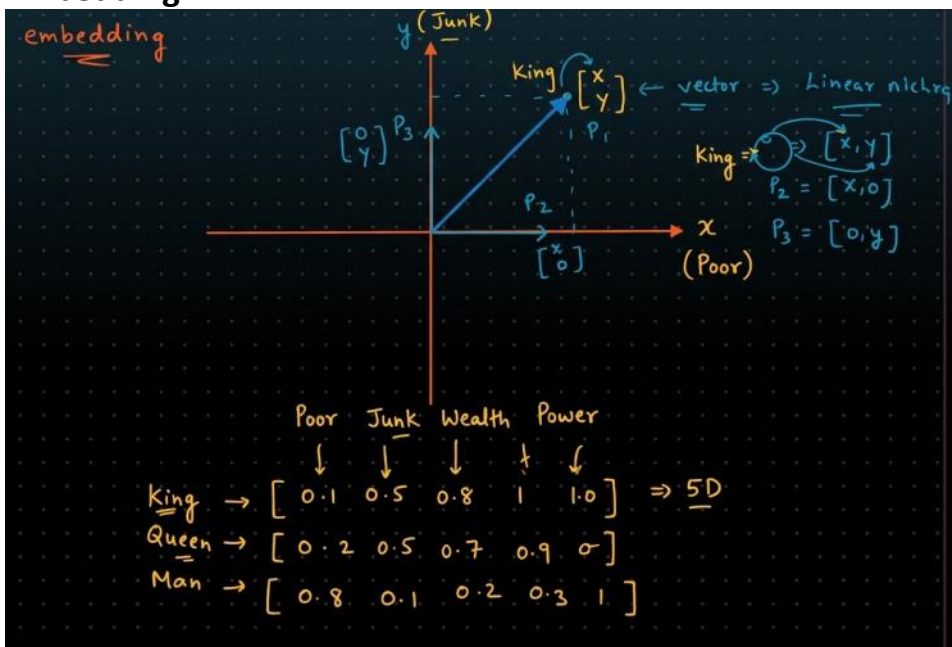


Retrieval - Obtaining the data from Database

Augment - Engaging information using LLM information knowledge base and Prompt

Generation - GPT, Gemini, Llama etc.

Embedding:



Similarity and Distance Methods:

similarity is determined using a **distance (or similarity) metric**. The metric tells us **how close two vectors (embeddings) are**. If the vectors are close, the meanings are considered similar.

Question: "What is Artificial Intelligence?"

Embedding: [0.25, 0.80, 0.45, 0.12]

Document: "AI is a branch of computer science."

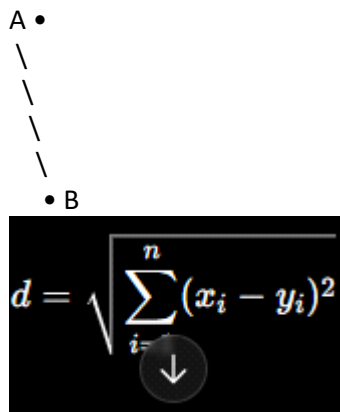
Embedding: [0.24, 0.79, 0.46, 0.11]

Since these vectors are very close, the document is highly relevant to the question.

Method	Used For	Best For
Euclidean Distance (L2)	Straight-line distance	Numerical data
Manhattan Distance (L1)	Grid/block distance	Sparse data
Cosine Similarity	Angle between vectors	NLP, embeddings, RAG
Dot Product	Magnitude + direction	Deep learning, Transformers

1. Euclidean Distance (Straight-Line Distance):

Imagine two houses on a field. The shortest path between them is a straight line.



two vectors:

A = (2,3)

B = (5,7)

Distance:

$$= \sqrt{((5-2)^2 + (7-3)^2)}$$

$$= \sqrt{(9+16)}$$

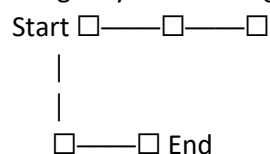
$$= \sqrt{25}$$

$$= 5$$

A **smaller Euclidean distance** means the vectors are more similar.

2. Manhattan Distance:

Imagine you're driving in a city like Manhattan. You can't go diagonally; you travel along streets.



$$d = \sum |x_i - y_i|$$

A = (2,3)

$$B = (5,7)$$

Distance

$$= |5-2| + |7-3|$$

$$= 3 + 4$$

$$= 7$$

3. Cosine Similarity:

Cosine similarity measures the **angle** between two vectors, not the physical distance. Imagine arrows pointing in directions.

Same direction

→

→

Cosine = **1.0** (Perfect match)

Slightly different

↗

→

Cosine $\approx$ **0.8** (Good match)

Perpendicular

↑

→

Cosine = **0** (No semantic similarity)

Opposite

←

→

Cosine = **-1** (Completely opposite)

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \|B\|}$$

where:

- $A \cdot B$ is the dot product.
- $\|A\|$ and $\|B\|$ are the vector lengths (magnitudes).

$$A = [1,2]$$

$$B = [2,4]$$

Dot Product:

$$1 \times 2 + 2 \times 4$$

$$= 10$$

Magnitude:

$$|A|$$

$$= \sqrt{1^2 + 2^2}$$

$$= 2.236$$

$$|B|$$

$$= \sqrt{2^2 + 4^2}$$

$$= 4.472$$

Cosine:

$$10 / (2.236 \times 4.472)$$

$$\approx 1$$

These vectors point in the same direction, so their cosine similarity is approximately 1.

4. Dot Product

$$A \cdot B = \sum x_i y_i$$

$$A = [2, 3]$$

$$B = [4, 5]$$

$$2 \times 4 + 3 \times 5$$

$$= 8 + 15$$

$$= 23$$

In Transformer attention, the dot product measures how strongly a **query** matches a **key**. A larger dot product generally indicates a stronger match before scaling and applying softmax.

Which Methods Are Used in Popular Vector Databases?

Vector Database	Similarity Method
FAISS	Cosine, L2, Inner Product
ChromaDB	Cosine, L2
Pinecone	Cosine, Dot Product, Euclidean
Milvus	Cosine, L2, Inner Product
Weaviate	Cosine
Qdrant	Cosine, Dot Product, Euclidean

Which One Should You Use?

Use Case	Recommended Metric
Semantic search (RAG) Sentence embeddings OpenAI, BGE, E5, Gemini embeddings	Cosine Similarity
Transformer attention	Dot Product (scaled)
Traditional ML with numeric features	Euclidean or Manhattan

Interview Summary

- **Euclidean Distance:** Measures the straight-line distance between two vectors.

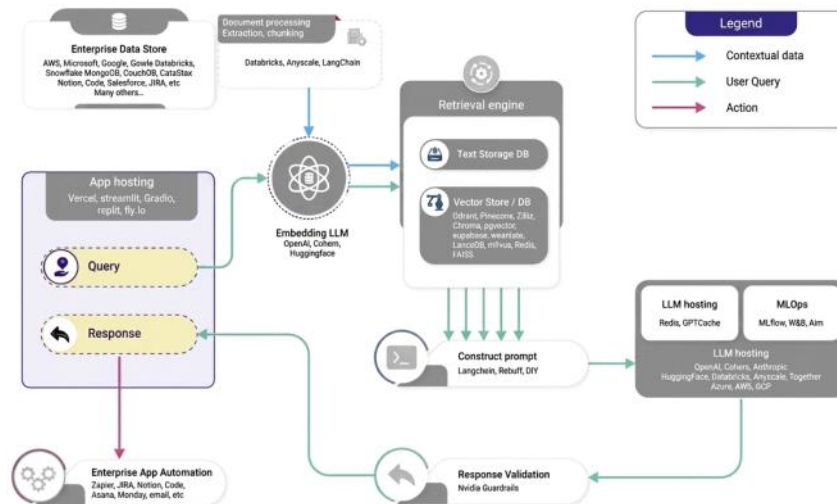
- **Manhattan Distance:** Measures the total distance traveled along each dimension.
- **Cosine Similarity:** Measures the angle between vectors; it is the most widely used metric for semantic search and RAG because it focuses on meaning rather than vector magnitude.
- **Dot Product:** Measures both direction and magnitude and is fundamental to the attention mechanism in Transformers.

Rule of thumb: Text → Embeddings → Cosine Similarity → Top-K Retrieval → LLM Generates Answer.

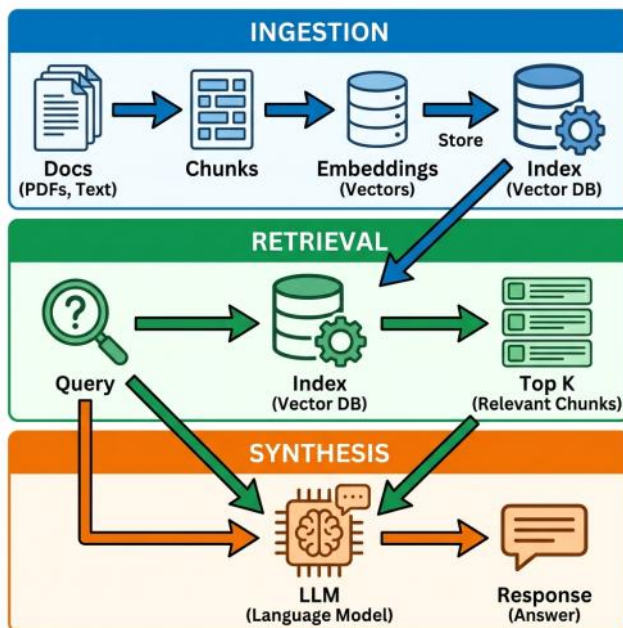
3. End-to-End RAG Pipeline:

RAG Flow: A Step by Step Representation

Figure 1 demonstrates the steps involved in building a RAG pipeline:



A) Let's understand the Ingestion process:



Step 1: Documents:

Imagine a company has (CSV, TSV, JSON, PDFs, DOCs, Web pages, XML, Database, Cloud Storage, Any Remote Location etc)

These are called **raw documents** (Employee Handbook.pdf, HR Policy.pdf, Leave Policy.pdf, Medical Insurance.pdf, Holiday Calendar.pdf, Travel Policy.pdf)

At this stage they are simply files.

The AI cannot understand PDFs directly.

Think of them as books written in different languages and formats.

Step 2: Document Loading

The system opens every document.

The loader extracts the text.

Now the PDF becomes plain text.

Step 3: Cleaning

The extracted text usually contains noise (Headers, Footers, Extra spaces, Blank lines, HTML tags, OCR mistakes (sometimes)). Now the document becomes clean.

Step 4: Chunking : We collect the data from various sources, split the data into the chunks.

Check it out: <https://www.pinecone.io/learn/chunking-strategies/>

Imagine your HR handbook has: 300 Pages

Will we convert all 300 pages into one embedding? No

Why?

Because embedding models have input length limits, and very large chunks mix many topics together, making retrieval less accurate.

Instead, we divide the document into smaller pieces called **chunks**.

Suppose this page:

Employees receive

12 Casual Leaves

10 Sick Leaves

5 Optional Holidays

Carry forward is allowed.

Unused leave expires after one year.

Chunk 1: Employees receive 12 Casual Leaves 10 Sick Leaves

Chunk 2: 5 Optional Holidays Carry forward is allowed.

Chunk 3: Unused leave expires after one year.

Instead of storing one huge paragraph, we store many small paragraphs.

Why Chunking?

Imagine you have a **300-page HR policy PDF**.

topics like:

Page 1–50 Company Introduction

Page 51–90 Leave Policy

Page 91–140 Salary Policy

Page 141–180 Medical Insurance

Page 181–250 Travel Policy
Page 251–300 Retirement Policy

Now an employee asks: "How many casual leaves do employees get?"
Should the AI search all 300 pages as one document? No

Why?

Problem 1: Embedding Models Have Input Limits

Embedding models cannot process an unlimited amount of text.

Ex: 300-page PDF ↓ 500,000 characters

An embedding model might only accept a limited number of tokens (depending on the model).

So we must divide the document into smaller pieces.

Problem 2: One Embedding Represents One Meaning

Suppose you create **one embedding** for this entire document:

[Company Introduction](#)

[Leave Policy](#)

[Salary Policy](#)

[Insurance Policy](#)

[Travel Policy](#)

[Retirement Policy](#)

The embedding becomes a mixture of many topics.

Asking someone: "Describe this entire book in one sentence."

The answer will be very general. Instead of capturing one clear idea, the embedding represents everything at once.

Analogy: Imagine a supermarket.

You have: Milk, Rice, Soap, Chocolate, Shampoo, Sugar

Would you put everything into one giant box?

📦 Milk, Rice, Soap, Chocolate, Shampoo, Sugar

Asking someone: "Can you bring me the soap?"

You now have to search through the whole box.

Instead, you organize items: 📦 **Dairy:** Milk, 📦 **Groceries:** [Rice, Sugar], 📦 **Personal Care:** [Soap, Shampoo]

Now finding the soap is quick. Chunking does exactly this with text.

Problem 3: Better Semantic Meaning

Suppose a document says: Employees receive 12 Casual Leaves 10 Sick Leaves

This paragraph talks only about **leave policy**.

embedding becomes: Leave Policy Meaning

Another paragraph says: Employees receive Medical Insurance Family Coverage
This embedding becomes: Insurance Meaning

Each chunk has a **clear semantic meaning**. That makes similarity search much more accurate.

Problem 4: Faster Search

Imagine a library with **1 million books**.

Would you search every page of every book? No.

You search the correct chapter.

Chunking makes the search much faster because the vector database compares the query against small, focused pieces instead of huge documents.

Problem 5: Lower Cost

Suppose: Entire PDF 300 pages

Sending all 300 pages to the LLM for every question is expensive because LLMs charge based on the number of input tokens.

Instead: Only 2 chunks ↓ About 500 words

This is much cheaper and faster.

Problem 6: Better Answer Quality

Analogy: Imagine a huge pizza 🍕

Can everyone eat the whole pizza? No

So we cut it into slices = 🍕 → 🍕 🍕 🍕 🍕 🍕 🍕 Each slice is a **chunk**.

How to figuring out the ideal chunk size?

Based on these factor you can decide the size of chunk.

- Data characteristics
- Retriever constraints
- Memory and computational resource
- Task Requirements
- Experimentation
- Overlap consideration

[Imagine a Textbook](#)

Suppose you have a **1000-page Deep Learning textbook**.

Someone asks: "What is Batch Normalization?"

How should we divide the book?

Option 1: Very Large Chunks (100 pages)

Chunk 1 Pages 1–100

Chunk 2 Pages 101–200

Chunk 3 Pages 201–300

If Batch Normalization is on page 150, you'll retrieve **100 pages**.

Problems: Most of the retrieved text is irrelevant, Higher LLM cost, More noise, Lower answer quality

Option 2: Very Small Chunks (1 sentence)

Suppose the document says:

Batch normalization normalizes the output of a layer during training.

It reduces internal covariate shift.

This improves convergence speed.

It also allows higher learning rates.

If every sentence becomes a chunk, the retriever may retrieve only:

"It also allows higher learning rates."

The explanation is incomplete because the surrounding context is missing.

Option 3: Medium Chunks (Ideal):

Chunk 1

Batch normalization normalizes the output of a layer during training.

It reduces internal covariate shift.

This improves convergence speed.

It also allows higher learning rates.

Now the chunk contains one complete idea. This is what we want.

Chunk Sizes:

Document Type	Chunk Size	Overlap
FAQ	100–300 tokens	20–50 tokens
Blogs	300–500 tokens	50–100 tokens
Technical Documentation	500–800 tokens	50–100 tokens
Research Papers	800–1000 tokens	100–200 tokens
Books	500–1000 tokens	100–200 tokens
Source Code	Function/Class level	Small overlap if needed

How Companies Find the Ideal Chunk Size:

They don't guess—they **experiment**.

Experiment	Chunk Size	Overlap
------------	------------	---------

1	256	20
2	512	50
3	1024	100

Measure: Retrieval accuracy, LLM answer quality, Latency, Cost

Compare all three.

Choose the configuration that gives the best balance of:

- Retrieval quality
- Response accuracy
- Speed
- Cost

This process is called **RAG evaluation** or **retrieval benchmarking**.

How do you determine the ideal chunk size in RAG?

There is no fixed chunk size that works for every application. The ideal chunk size depends on:

1. **Document structure** (FAQs, books, legal contracts, research papers, etc.).
2. **The type of user queries** (short factual questions vs. long explanatory questions).
3. **The embedding model's capabilities** and context limits.
4. **Maintaining semantic completeness**—each chunk should represent one coherent topic.
5. **Empirical evaluation**, by testing different chunk sizes and overlaps and selecting the one that delivers the best retrieval accuracy, answer quality, latency, and cost.

Step 5: Embeddings

This is where AI begins to understand meaning.

Suppose one chunk says: Employees receive 12 Casual Leaves.

Humans understand this sentence.

Computers don't.

So we convert it into numbers.

Example:

Sentence: [Employees receive, 12 Casual Leaves]

Embedding: [0.21, 0.84, 0.31, 0.66, 0.91, 768 numbers]

1536, 3072 depending on the embedding model.

These numbers are called an **embedding vector**.

Types of Embeddings Based on Technique:

Frequency-Based Methods (Old)

- One-Hot Encoding
- Bag of Words (BoW)
- TF (Term Frequency)
- TF-IDF

Prediction-Based Embeddings

- Word2Vec
- GloVe
- FastText

Contextual Embeddings

- BERT
- SBERT

- OpenAI Embeddings
- BGE
- E5
- Gemini Embeddings

Modern Embedding Models:

Model	Company	Primary Use
Word2Vec	Google	Word Embeddings
GloVe	Stanford	Word Embeddings
FastText	Meta	Word Embeddings
BERT	Google	Contextual Word Embeddings
SBERT	Sentence Transformers	Sentence Embeddings
BGE	BAAI	RAG, Retrieval
E5	Microsoft	Retrieval
OpenAI Embeddings	OpenAI	RAG, Semantic Search
Gemini Embeddings	Google	RAG, Search
CLIP	OpenAI	Text + Image Embeddings

Token level embedding vs Sentence level embedding

Checkout the link: <https://huggingface.co/sentence-transformers/>
<https://www.sbert.net/index.html>

Which embedding model you need to select:

Checkout the leader board: <https://huggingface.co/spaces/mteb/leaderboard>

Step 6: Store in Vector Database: FAISS, ChromaDB, Pinecone, Milvus, Weaviate, Qdrant

The retriever here could be any of the following depending on the need

Vector DB: A vector database indexes and stores vector embeddings for fast retrieval and similarity search, with capabilities like CRUD operations, metadata filtering, horizontal scaling and serverless.

Graph DB: Graph databases are designed to represent and store data as graphs. This make it easy to represent people, products, and events along with what ties them together. Search engines, logistics businesses, and social networks typically use graph databases to understand connection in their data.

Nodes are the primary entities in a graph database. Each node holds all data about a person, product, business, event, or another entity.

Edges are the connecting parts of graph databases. They show similarities, relationships, and commonalities. You can define the properties and weights of edges to fit your purpose.

Regular SQL database: Offers structure data storage and retrieval but might lack the semantic flexibility of vector databases.

Graph databases bring you the full power of relationships in data.

Vector databases are best suited for managing and querying high-dimensional data in use cases that require similarity searches.

It can be hard to make the choice and go with either graph or vector technology for your database.

With generative AI, large language model (LLM), and real-time data playing an increasing part in modern applications, we're seeing an increase in combined solutions.

This is why **Neo4j** recently added the ability to perform vector similarity search. They aim to make more sense of data and combat LLM hallucinations by blending similar feature vectors to input vectors found through lookups in the knowledge graph.

Checkout the link: <https://docs.datastax.com/en/astra-db-serverless/api-reference/table-index-methods/create-vector-index.html>

Step 7: Indexing: HNSW (Hierarchical Navigable Small World), IVF (Inverted File Index), PQ (Product Quantization),

These are all examples of **Approximate Nearest Neighbour (ANN)** search techniques.